Does Spec-Driven Development Reduce Defects? An Empirical Test of Industry Claims Across 119 Open-Source Repositories

Brenn Hill

Independent Researcher

Working Paper — April 2026

Suggested citation: Hill, B. (2026). Does Spec-Driven Development Reduce Defects? An Empirical Test of Industry Claims Across 119 Open-Source Repositories. SSRN Working Paper.

Abstract

Spec-driven development (SDD) tools — GitHub's Spec Kit, Amazon's Kiro, and others — claim that writing specifications before implementation reduces defects, prevents rework, and improves code quality. These claims lack empirical evidence. We provide the first large-scale test: 88,052 pull requests (after bot exclusion) across 119 open-source repositories, using SZZ defect tracing and within-author fixed-effects estimation. We test specification *artifacts* in open-source pull requests — the closest available proxy for SDD workflows, scored on the same quality dimensions SDD tools prescribe — not the tools' integrated workflows directly. Five hypotheses derived from vendor claims are tested. Four of five are not supported. The naive association between specifications and defects is *reversed* (specifications accompany more bugs, not fewer); after within-author controls, the most likely interpretation is confounding by indication: harder tasks receive specifications and independently produce more defects. Specification quality has no meaningful effect on rework ($p = 0.687$). One claim finds partial support: specifications do constrain AI-generated code scope ($p = 0.001$), though this effect does not translate into fewer defects or less rework. Nine robustness checks confirm the quality null. Specification artifacts proxy for task complexity, not quality improvement.

1 Introduction

The rise of AI-assisted software development has produced a new category of developer tooling: specification-driven development (SDD) frameworks. These tools — including GitHub's Spec Kit (Delimarsky, 2025), Amazon's Kiro (Brooker, 2025), and Tessl — claim that writing formal specifications before implementation leads to fewer defects, less rework, and higher-quality code, particularly when AI agents perform the implementation.

Böckeler (Böckeler, 2025) defines spec-driven development as "writing a 'spec' before writing code with AI" where "[t]he spec becomes the source of truth for the human and the AI." The GitHub Copilot Academy frames it more strongly: "Specifications don't serve code — code serves specifications" (GitHub Copilot Academy, 2025). The movement gained momentum in 2025 with the release of GitHub's Spec Kit (open-source, v0.0.72 as of this writing) and Amazon's Kiro IDE. Böckeler examined three tools pursuing the approach (Böckeler, 2025). SDD is positioned as an antidote to "vibe coding" — iteratively prompting AI without formal requirements — which practitioners associate with scope creep, rework, and defects.

The core premise is intuitive: if developers articulate what software should do before writing it, AI systems will produce more reliable output. The GitHub Copilot Academy lists "reduced rework, living documentation, systematic quality" among SDD's benefits (GitHub Copilot Academy, 2025). Kiro claims that specifications enable "fewer iterations and higher accuracy" and "prevent costly rewrites" (Yanchuleff, 2025). GitHub's Spec Kit blog post promises "less guesswork, fewer surprises, and higher-quality code" (Delimarsky, 2025).

These claims rest on a plausible causal chain: clearer requirements → less ambiguity → fewer defects. However, none of the marketing materials, documentation, or blog posts accompanying these tools cite empirical evidence for the claimed outcomes. The assertions are presented as self-evident truths rather than testable hypotheses. No study has tested SDD's quality claims on production-scale data prior to this paper.

This paper provides the first large-scale empirical test. We analyze 88,052 merged pull requests (after excluding 12,195 bot-authored PRs) across 119 open-source repositories, using the SZZ algorithm to trace defects back to their introducing commits and within-author fixed-effects estimation to control for developer-level confounding. We derive five hypotheses directly from vendor claims and test each with the most conservative methodology available for observational data.

2 Related Work

2.1 Requirements Engineering and Defect Prevention

The claim that upfront specification prevents defects has deep roots in software engineering. Boehm's cost-of-change curve (Boehm, 1981) argued that requirements defects are exponentially cheaper to fix early. However, subsequent work has challenged the universality of this finding. Shull et al. (Shull et al., 2002) found that the cost multiplier depends heavily on project context, and agile methodologies have demonstrated that iterative refinement can be more effective than upfront specification for certain project types (Beck & al., 2001).

The question of whether requirements quality predicts software defects has been studied directly. Mund et al. (Mund et al., 2015) combined two empirical studies and found only weak evidence that requirements specification quality predicts implementation defects, with effect sizes that varied substantially across projects. Femmer et al. (Femmer et al., 2017) developed automated detection of "requirements smells" (ambiguity, incompleteness, inconsistency) and found that while smells were prevalent, their relationship to downstream defects was context-dependent. These findings suggest that the relationship between specification quality and code quality is not as straightforward as SDD vendors assume — a pattern our study confirms at larger scale.

2.2 SDD Literature

The academic literature on SDD perpetuates the pattern of unsubstantiated claims. Piskala (Piskala, 2026) claims "controlled studies showing error reductions of up to 50%," citing a Red Hat Developer blog post (Naszcyniec, 2025) and an InfoQ article (Griffin & Carroll, 2026) as evidence. Neither source contains any controlled study, any experiment, or any quantitative data. The "50%" figure traces through a citation chain that terminates in practitioner opinion pieces with no empirical backing. Guo et al. (Guo et al., 2025) claim SDD "guarantees software quality" without empirical validation. Marri (Marri, 2026) reports a 73% security defect reduction from constitutional specs, but on a single project ($n = 1$), with the same developer implementing both conditions sequentially — the author acknowledges "small sample size limits statistical power." Roy (Roy, 2026) proposes iterative spec-code co-evolution but reports no independently validated quality outcomes. Zietsman (Zietsman, 2026) runs small planted-bug experiments and is honest about their limitations ("directional, not statistically significant"), while contributing the useful observation that AI reviewers without specs suffer circular reasoning — but does not test whether specs reduce defects at scale.

Practitioner evaluations have been more skeptical. Eberhardt, writing for Scott Logic, found Spec Kit roughly ten times slower than iterative development, producing thousands of lines of specification markdown that still resulted in buggy code (Eberhardt, 2025). Zaninotto argued that "SDD shines when starting a new project from scratch, but as the application grows, the specs miss the point more often and slow development," and proposed iterative natural-language development as a faster alternative (Zaninotto, 2025).

No study has tested SDD's quality claims on production-scale data prior to this paper.

3 Study Design

3.1 Hypotheses

We identified five empirical claims made by SDD tool vendors in their public documentation, blog posts, and marketing materials. Each claim is stated verbatim and mapped to a testable hypothesis.

Claim 1: Specifications reduce defects.

GitHub's Spec Kit blog post promises "less guesswork, fewer surprises, and higher-quality code" (Delimarsky, 2025). Kiro claims specifications lead to "fewer iterations and higher accuracy" (Yanchuleff, 2025).

H1: Pull requests with specification artifacts have lower defect-introduction rates than pull requests without them, controlling for author and PR size.

Claim 2: Specifications reduce rework.

The GitHub Copilot Academy lists "reduced rework" among SDD's benefits (GitHub Copilot Academy, 2025). Kiro claims to "prevent costly rewrites" (Yanchuleff, 2025). GitHub's blog states that SDD enables "iterative development without expensive rewrites" (Delimarsky, 2025).

H2: Pull requests with specification artifacts have lower rework rates than pull requests without them, controlling for author and PR size.

Claim 3: Clearer specifications produce more reliable code.

GitHub's Spec Kit blog states that "providing a clear specification up front, along with a technical plan and focused tasks, gives the coding agent more clarity, improving its overall efficacy" (Delimarsky, 2025). Kiro claims specifications enable "fewer iterations and higher accuracy" (Yanchuleff, 2025). This implies a dose–response relationship: higher specification quality should predict fewer defects.

H3: Among pull requests with specifications, higher specification quality predicts lower defect-introduction rates, controlling for author and PR size.

H4: Among pull requests with specifications, higher specification quality predicts lower rework rates, controlling for author and PR size.

Claim 4: Specifications constrain AI-generated code scope.

GitHub's Spec Kit blog describes tasks that give "the coding agent a way to validate its work and stay on track" (Delimarsky, 2025). Kiro claims specs provide "a North Star to guide the work of the agent, allowing it to take on larger tasks without getting lost" (Brooker, 2025).

H5: Specifications reduce code churn (additions + deletions) for AI-tagged pull requests more than for human-authored pull requests, controlling for author.

3.2 Dataset

We collected pull request data from 119 open-source repositories spanning 14 programming languages. Repositories were selected to represent large, active projects with diverse contribution patterns. We collected data between March 28 and April 3, 2026, with a 365-day lookback window per repository. The resulting dataset spans April 2025 through early April 2026 (p1–p99: April 8, 2025 to April 1, 2026), totaling 100,247 PRs. We excluded 12,195 bot-authored PRs (dependabot, renovate, and other automated accounts identified by username pattern and bot flags), leaving 88,052 PRs from human and AI-assisted authors.

For each pull request, we extracted: author, merge date, additions, deletions, files changed, linked issues, PR description, review comments, and co-author tags (used to identify AI-assisted contributions).

Metric	**Value**	**Notes**
Repositories	119	convenience sample, 17 languages
PRs (post-bot exclusion)	88,052	12,195 bots removed
Median PRs per repo	517	range 9–4,803
Median spec rate (per repo)	0.0%	IQR 0.0–15.3%
Median AI tag rate (per repo)	0.0%	IQR 0.0–0.9%
SZZ-covered repos	103	16 had unreachable merge SHAs
SZZ bug-introducing PRs	9,754	12.6% of SZZ-covered PRs
Quality-scored PRs	5,192	5.9% of total; non-random subsample

Table 1: Study dataset overview.

The sample spans 17 programming languages, with Python (31 repos, 26%), TypeScript (26, 22%), Go (17, 14%), and Rust (13, 11%) predominating. Specification rates are highly skewed: the median repository has 0% spec'd PRs, while the 75th percentile is 15.3%. AI tagging is concentrated: 76 of 119 repositories have zero AI-tagged PRs, and the median AI tag rate is 0% (75th percentile: 0.9%). However, the within-author and within-repo designs compare AI and non-AI work within the same contexts, and the 5,989 AI-tagged PRs provide sufficient statistical power for the primary AI analyses. We recognize that the AI-specific results (H5, AI subgroups) would benefit from a larger AI-tagged sample as agentic workflows become more prevalent.

3.3 Operationalization

3.3.1 Specification Presence (`specd`)

A pull request is classified as "specified" if it meets any of the following criteria: (a) it links to a GitHub issue or external tracking ticket, (b) its description references a specification document, RFC, or design doc, or (c) its description contains structured requirements content (acceptance criteria, behavioral descriptions, scope boundaries). Classification is performed programmatically by parsing PR metadata: issue references (`#123`, `fixes #456`), cross-reference URLs, and description structure (presence of headings, checklists, or requirement-style language). 7,837 of 88,052 PRs (8.9%) are classified as specified.

This operationalization is deliberately generous: it captures any form of upfront specification artifact, from a linked Jira ticket to a detailed RFC. SDD tools claim benefits from *any* form of specification, so we test the broadest reasonable definition. Specification *presence* and specification *quality* are tested separately: H1 and H2 test whether having any specification artifact predicts fewer defects or less rework; H3 and H4 test whether, among the subset of PRs that have specifications, higher-quality specifications predict better outcomes. This two-level design addresses the objection that our binary `specd` measure is too coarse by also testing whether the content of the specification matters.

3.3.2 Specification Quality (`q_overall`)

For the subset of specified PRs with substantial descriptions (body length > 50 characters), we scored specification quality by prompting Claude Haiku (Anthropic) with the PR title and body and a structured rubric. The LLM scored each PR across seven dimensions: outcome clarity, error states, scope boundaries, acceptance criteria, data contracts, dependency context, and behavioral specificity. Each dimension is scored 0–100, and the overall quality score is the mean across dimensions.

These seven dimensions were chosen to align with the quality criteria recommended by SDD tools themselves. Spec Kit's template prescribes "core requirements" (outcome clarity), "acceptance criteria" (our acceptance criteria dimension), "edge cases" (error states), "data models" and "API contracts" (data contracts), and "user stories" (behavioral specificity) (Delimarsky, 2025). Kiro prescribes "clear requirements," "acceptance criteria," "system architecture," and "technology choices" (Yanchuleff, 2025). Six of our seven dimensions have direct equivalents in Spec Kit's recommended structure; all seven are covered across both tools. This alignment is deliberate: we measure specification quality on the dimensions that SDD vendors themselves say matter.

We validated the LLM scoring against independent human ratings on a stratified random sample of 30 PRs. The human rater scored each dimension on the same rubric, blind to the LLM scores. Overall rank-order agreement was moderate: Spearman $\rho = 0.37$ ($p = 0.04$), Pearson $r = 0.44$ ($p = 0.02$). The LLM scores systematically higher than the human rater (mean bias: +7 points on the 0–100 scale, mean absolute difference: 16 points). Per-dimension agreement varies:

scope boundaries ($\rho = 0.45$, $p = 0.01$), dependency context ($\rho = 0.37$, $p = 0.04$), and behavioral specificity ($\rho = 0.39$, $p = 0.03$) show significant agreement, while acceptance criteria ($\rho = 0.02$) and error states ($\rho = 0.18$) show near-zero correlation. The LLM reliably detects whether specification content is *present* but cannot judge whether it is *correct* for the domain — a distinction between formal completeness and functional quality. Since even this completeness-oriented measure shows no defect benefit, the null finding is conservative: a more precise quality measure would sharpen the estimate, not reverse it.

5,192 PRs (5.9% of total) received quality scores. This subsample is non-random — only spec'd PRs with sufficient description text were scored — and results from this subsample carry a selection bias caveat.

A further limitation: we cannot distinguish specifications written before implementation from descriptions written after. Many PR descriptions in our dataset are post-hoc — the developer wrote the code, validated it, possibly iterated through review, and then described what it does. These post-hoc descriptions should score *higher* on our quality rubric than pre-implementation specifications, because the developer is describing validated behavior rather than predicting it. The fact that even this generous measure — which includes descriptions written with full knowledge of what the code actually does — shows no defect-prevention benefit makes the SDD claim harder to sustain, not easier. If describing what code *does* after building it does not predict fewer defects, describing what code *should do* before building it is unlikely to fare better.

Furthermore, many effective AI instructions are inherently underspecified by our rubric's standards. A prompt like "improve the error handling, it looks ugly" scores low on outcome clarity, acceptance criteria, and behavioral specificity — yet it is a perfectly successful instruction for an AI agent that can read the surrounding code. The agent does not need the spec to enumerate every error case; it can infer them from context. This class of task — where the codebase itself provides sufficient specification — is invisible to our quality measure but may represent a large share of productive AI-assisted work. If detailed specifications add no value for tasks where the code context is sufficient, the marginal benefit of specification effort is even smaller than our results suggest.

3.3.3 Defect Introduction (`szz_buggy`)

We applied the SZZ algorithm (Śliwerski et al., 2005) to 103 of 119 repositories, tracing 64,805 blame links from fix commits to bug-introducing commits. The remaining 16 repositories produced zero traceable blame links: their merge commit SHAs (recorded by the GitHub API) are not reachable in single-branch clones, typically because the repository uses squash-merge workflows where GitHub's synthetic merge commits are garbage-collected after merging. The SZZ algorithm traces defect-fixing commits back to the commits that introduced the defect using `git blame`. We use the basic SZZ variant, which da Costa et al. (Costa et al., 2017) found misattributes 46–71% of bug-introducing changes depending on the project. This noise is substantial, but it is non-differential with respect to specification presence: there is no reason to expect SZZ to systematically misattribute more for spec'd PRs than unspec'd PRs. Non-differential measurement error attenuates associations toward the null, which works against finding a protective effect but

does not create spurious reversed associations. We acknowledge the implications for statistical power in Section 6. Each bug-introducing commit was mapped to its originating pull request. A PR is marked `szz_buggy = True` if any of its commits were identified as introducing a defect that was later fixed. 9,754 PRs (12.6% of those in SZZ-covered repos) are marked as bug-introducing.

3.3.4 Rework (`reworked`)

A pull request is classified as reworked if a subsequent PR by any author modifies overlapping files and has a title or description indicating correction (matching patterns such as "fix," "revert," "bugfix," "hotfix," "regression," "broke," or "broken"). File overlap is computed by comparing the set of files changed in each PR pair. 3,890 PRs (3.9%) are classified as reworked.

3.3.5 AI-Tagged (`ai_tagged`)

A pull request is classified as AI-assisted through two detection layers. First, a regex matches co-author tags (e.g., `Co-authored-by: Copilot`), generation attribution strings (`Generated with`, `Claude Code`, `written with/by` followed by a tool name), AI agent markers (`CURSOR_AGENT`, `coderabbit.ai`, `AI-generated`), and the robot emoji (🤖). Second, we parse structured "AI Disclosure" and "AI Usage" sections that some repositories include in their PR templates, classifying the disclosure content as AI-used or no-AI-used. We validated the combined classifier against 537 PRs with clear self-reported AI disclosures: precision is 98.8% (1 false positive) and recall is 24.2% (75.8% false negatives). The low recall reflects that many developers describe AI use in natural language that our patterns do not capture. 5,989 PRs (6.8%) carry AI tags after bot exclusion. This remains a lower bound.

3.3.6 Code Churn (`total_churn`)

Total code churn is defined as additions + deletions. We use `log(1 + churn)` in regressions to normalize the heavily right-skewed distribution (median: 55, mean: 698).

3.4 Statistical Approach

For each hypothesis, we report three levels of analysis:

Pooled: Raw rates and Fisher's exact test. This is the analysis that naive claims would cite.

Controlled: Logistic regression with size controls (log additions, log deletions, log files changed) and repository fixed effects (dummy variables). This addresses between-repository confounding.

Within-author: Linear probability model (LPM) with full author demeaning and clustered standard errors. All variables — treatment, controls, and outcome — are demeaned by author, equivalent to author fixed effects via the Frisch–Waugh–Lovell theorem. Standard errors are clustered at the author level to account for within-author residual correlation. We use LPM rather than logistic regression because demeaning is exact for OLS but produces biased estimates in logit due to the incidental parameters problem (Neyman & Scott, 1948). This follows Angrist and

Pischke's recommendation for fixed-effects estimation with binary outcomes (Angrist & Pischke, 2009). Primary models use size controls (log additions, log deletions, log files changed); we verify that results hold when adding JIT defect prediction features (Kamei et al., 2013) as additional controls (Section 4.7).

The within-author estimate is the most credible. Observational studies of developer practices face severe confounding: developers who write specifications may differ systematically from those who do not in skill, experience, task selection, and organizational context. The within-author design addresses this by comparing the same developer's specified PRs to their own unspecified PRs, eliminating all time-invariant author-level confounding. The treatment effect is identified only from authors who have *both* specified and unspecified PRs in the dataset.

We restrict within-author analysis to authors with ≥2 PRs and report the number of authors with treatment variation (those who have both spec'd and unspec'd PRs), as these are the observations that identify the effect.

We do not apply multiple-comparison corrections across the five hypotheses. At $\alpha = 0.05$, approximately 0.25 false positives are expected by chance. We interpret all results with this in mind.

4 Results

4.1 H1: Specifications Reduce Defects

Analysis restricted to 77,502 PRs in the 103 repositories with SZZ coverage.

Method	Coefficient	*p*-value	Interpretation
Pooled (Fisher's exact)	OR = 1.247	< 0.001	Spec'd PRs have *higher* bug rate
Controlled (logit + repo FE)	0.098	0.006	Effect persists with size + repo controls
Within-author (LPM)	+0.017	0.021	+1.7pp *increase* in bug rate

Identified from 894 authors with treatment variation (out of 4,328 authors with ≥2 PRs).

H1 is not supported. The table should be read bottom-up. The pooled comparison (top row) is the naive analysis: spec'd PRs have a 24.7% higher odds of introducing a bug than unspec'd PRs. But this comparison is confounded — developers who write specs are different from those who don't, and the repos that encourage specs are different from those that don't. The controlled logistic regression (middle row) adds size controls and repository fixed effects, removing between-repo confounding. The effect persists.

The within-author estimate (bottom row) is the most credible. It compares the *same developer's* spec'd PRs to their own unspec'd PRs, eliminating all time-invariant author-level confounding — skill, experience, coding style, and organizational context are held constant because both the "treatment" and "control" come from the same person. The result: within-author, spec'd

PRs are associated with a 1.7 percentage-point *increase* in defect-introduction rates ($p = 0.021$). The direction is opposite to the vendor claim at every level of analysis.

This is consistent with confounding by indication: harder, riskier tasks receive specifications *and* introduce more defects. A developer rationally invests specification effort in proportion to task complexity. The spec does not cause the bugs — the task difficulty that motivates the spec also produces the bugs. When JIT risk features are added as controls (Section 4.7), the within-author spec coefficient drops 82% and loses significance ($p = 0.517$), confirming that measurable task complexity explains most of the association.

A plausible alternative explanation is detection bias: if spec'd PRs have better test coverage and more granular QA, their bugs may be detected and fixed individually, making them more visible to SZZ tracing. Unspec'd PRs might have bugs that go undetected longer or get fixed in bulk refactoring commits that SZZ cannot trace, producing artificially low bug rates. We cannot rule this out entirely. However, two observations weigh against it. First, the rework measure (H2) does not depend on SZZ tracing and shows the same directional pattern. Second, propensity score matching (Section 4.8) eliminates the defect association entirely by matching on observable risk features. This suggests task complexity — not detection asymmetry — is the primary driver.

4.2 H2: Specifications Reduce Rework

Analysis includes all 88,052 PRs.

Method	Coefficient	*p*-value	Interpretation
Pooled (Fisher's exact)	OR = 5.915	< 0.001	Spec'd PRs have *5.9×* higher rework
Controlled (logit, no repo FE[†])	0.366	< 0.001	Effect persists
Within-author (LPM)	+0.049	< 0.001	+4.9pp *increase* in rework rate

[†] Repository fixed effects caused a singular matrix; controlled estimate uses size controls only.

Identified from 985 authors with treatment variation (out of 4,892 with ≥2 PRs).

H2 is not supported. The within-author effect is strong and in the wrong direction: the same author's spec'd PRs have a 4.9 percentage-point higher rework rate than their unspec'd PRs. The pooled odds ratio (6.1×) dramatically overstates the association — most of the pooled effect is between-author confounding — but even within-author, specs are associated with more rework, not less.

4.3 H3: Specification Quality Reduces Defects

H1 and H2 tested specification *presence* — whether having any specification artifact matters. H3 and H4 test specification *quality* — whether, among PRs that already have specifications, better-written specifications predict better outcomes. This is the stronger version of the SDD claim: not just "write a spec" but "write a *good* spec."

Quality is the mean of seven rubric dimensions (described in Section 3.3), each scored 0–100. The overall quality score is the mean across dimensions. These dimensions were chosen to align with the quality criteria that SDD tools themselves prescribe (Section 3.3). The scoring is automated and has not been validated against human expert judgment — a limitation we acknowledge (Section 6). The quality score measures *formal* specification completeness (are error states described? are acceptance criteria present?) rather than *functional* correctness (are the right error states described? are the acceptance criteria valid for this domain?). A specification can score highly on every dimension and still specify the wrong behavior.

Analysis restricted to 5,246 PRs with LLM-scored specification quality (6.6% of SZZ-covered PRs). Selection bias caveat: only specified PRs with substantial descriptions were scored.

Method	Coefficient	p-value	Interpretation
Controlled (logit + repo FE)	−0.005	0.042	Quality predicts fewer bugs
Within-author (LPM)	−0.001	0.018	−0.08pp per quality point

Identified from 358 authors with treatment variation.

H3 is marginally supported on a biased subsample. The within-author coefficient is statistically significant but substantively negligible: a 10-point improvement in specification quality (the full scale range) predicts a 0.8 percentage-point reduction in defect rate. This is identified from only 358 authors on 5.9% of the data, in a subsample that was non-randomly selected for having substantial specification content. We do not consider this finding robust.

4.4 H4: Specification Quality Reduces Rework

Method	Coefficient	p-value	Interpretation
Controlled (logit + repo FE)	0.003	0.253	Quality does not predict rework
Within-author (LPM)	−0.000	0.687	Effectively zero

Identified from 357 authors with treatment variation.

H4 is not supported. Specification quality has no meaningful relationship with rework rates within-author. The coefficient is effectively zero, with $p = 0.687$. The controlled logistic regression is also non-significant ($p = 0.328$), consistent with the within-author finding (thorough specifications accompany difficult tasks that are more likely to be reworked).

4.5 H5: Specifications Constrain AI Scope

Analysis uses log code churn (additions + deletions) as outcome. PRs with zero churn excluded (missing data artifact). AI detection combines regex pattern matching with structured AI disclosure parsing (Section 3.3), identifying 5,989 AI-tagged PRs — 2.3× more than co-author tags alone.

Group	Within-author coef	p-value	Interpretation
AI-tagged PRs	+0.042	0.561	No scope constraint for AI alone
Human PRs	+0.128	< 0.001	Spec'd human PRs are *larger*
Interaction (spec × AI)	−0.198	0.001	Specs constrain AI scope *relative to human*

AI analysis identified from 140 authors with treatment variation; interaction from 202.

H5 is partially supported. The interaction term is significant ($p = 0.001$): specifications reduce code churn for AI-tagged PRs *more* than for human PRs. Within-author, spec'd human PRs are 12.8% larger than unspec'd human PRs (the confounding-by-indication pattern: harder tasks get specs). But spec'd AI PRs are only 4.2% larger, and the differential effect is −19.8 percentage points. Specifications appear to constrain the scope expansion that AI otherwise produces.

A caveat: we measure code churn in the merged PR, not the total churn during development. We cannot observe how much iteration and discarded generation occurred before the PR was opened. The scope constraint we detect could reflect either upfront specification reducing unnecessary generation, or post-hoc curation producing a tighter final PR. However, AI code generation is fast enough that the pre-PR iteration cost is low — and the result is consistent with the intuition that clarity of intent reduces needless generation, regardless of when that clarity is established.

This scope constraint does not translate into quality improvement. AI-tagged spec'd PRs still show no defect benefit ($p = 0.628$ for bugs, $p = 0.138$ for rework in the subgroup analysis). The specification constrains *how much* the AI generates but not *how correctly* it generates. Scope constraint is a necessary but not sufficient condition for the quality improvements vendors claim.

4.6 Robustness Checks

Nine additional analyses test whether the null result is an artifact of our primary measures or insufficient controls.

4.6.1 Alternative Outcome Measures

In addition to SZZ-traced defects and rework, we test specification effects on two additional outcome measures: *escaped defects* (PRs whose CI passed but were subsequently followed by a fix or revert) and *strict escaped* (escaped PRs where the follow-up PR's title explicitly indicates a fix or regression).

Outcome	Within-author coef	p-value	N
SZZ bugs	+0.019	0.011	73,276
Rework	+0.050	< 0.001	83,188
Escaped	−0.002	0.169	83,188
Strict escaped	−0.000	0.870	83,188

The SZZ bugs and rework rows reproduce the H1 and H2 findings (included here for comparison). Both show significant *positive* associations — specs accompany more bugs and more rework, not less — which we attribute to confounding by indication (Section 5.2): harder tasks receive specifications and independently produce more defects and rework. The escaped and strict escaped measures, which capture a different class of defect (CI-passing code that subsequently required a fix), are directionally protective but not significant. No outcome measure shows specifications reducing defects at $p < 0.05$.

4.6.2 Incremental Validity Beyond JIT Features

The JIT defect prediction framework (Kamei et al., 2013) uses 14 code-change features (subsystems touched, file entropy, developer experience, etc.) to predict defect-introducing commits. We test whether specification information adds predictive power beyond JIT features on the 62,126 PRs with both complete JIT features and SZZ outcomes.

Model	**Pseudo R^2**	**AIC**	**Spec variable p**
JIT features only	0.075	45,850	—
JIT + spec presence	0.075	45,851	0.250
JIT + spec quality[†]	0.094	—	0.044

[†] Scored subset only (5,192 PRs). JIT-only Pseudo R^2 on same subset = 0.093.

Spec presence adds nothing to the JIT model: Δ Pseudo $R^2 = 0.00003$, $p = 0.250$, and AIC increases (worse fit). Spec quality adds a marginally significant but substantively negligible increment ($\Delta\ R^2 = 0.001$, $p = 0.044$) on a biased subsample. JIT features alone account for all the predictive power that specifications claim to provide.

4.6.3 Individual Quality Dimensions

We test each of the seven specification quality dimensions independently against SZZ bugs and rework (within-author LPM, 4,955 PRs):

Dimension	**→ bugs coef**	**p**	**→ rework coef**	**p**
Outcome clarity	−0.001	0.140	−0.000	0.495
Error states	−0.000	0.074	+0.000	0.437
Scope boundaries	−0.001	0.033	−0.000	0.214
Acceptance criteria	−0.001	0.079	−0.000	0.879
Data contracts	−0.001	0.031	+0.000	0.602
Dependency context	−0.000	0.512	+0.000	0.461
Behavioral specificity	−0.001	0.023	−0.000	0.693

All coefficients in the bugs column are negative (directionally protective) but extremely small. To put the magnitudes in plain language: the largest coefficient is −0.001 (scope boundaries). This means that scoring the maximum possible improvement on scope boundaries — moving from 0 to 100 on the rubric — would predict 0.1 fewer percentage points of bugs. In a codebase with a 12% base bug rate, that is a reduction from 12.0% to 11.9%. No individual quality dimension produces a practically meaningful reduction in defects. The rework column is even starker: every coefficient rounds to zero.

Three dimensions show marginal statistical significance for bugs (scope boundaries $p = 0.033$, data contracts $p = 0.031$, behavioral specificity $p = 0.023$), but none survive Bonferroni correction[1] (threshold: $p < 0.007$ for 7 tests). Statistical significance at these magnitudes reflects the large sample size, not a meaningful effect. Even if these tiny coefficients reflected real causal relationships, they would be too small to justify the specification effort required to achieve them.

4.6.4 Repo-Level Analysis

As a final robustness check, we aggregate to the repository level ($N = 98$ repos with ≥50 PRs) and test whether repos with higher specification rates have lower defect or rework rates.

Comparison	**Spearman ρ**	**p-value**
Spec rate vs. bug rate	+0.091	0.371
Spec rate vs. rework rate	+0.919	< 0.001

At the repository level, specification rate has no relationship with bug rate ($\rho = 0.09$, $p = 0.37$) and a massive *positive* correlation with rework rate ($\rho = 0.92$, $p < 0.001$). The latter is confounding by indication at the repo level: projects that use specifications tend to be complex projects with higher rework rates. An OLS regression controlling for mean PR size shows the spec rate coefficient dropping to +0.025 ($p = 0.60$) — effectively zero.

4.6.5 Subgroup Analysis

SDD tools are marketed primarily for AI-assisted workflows. We test whether the null result holds across human-only PRs, AI-tagged PRs, and repos stratified by AI adoption rate. As in all analyses, bot PRs are excluded.

Subgroup	**→ bugs coef**	**p**	**→ rework coef**	**p**
All non-bot PRs (88,052)	+0.017	0.021	+0.049	< 0.001
Human-only, no AI (82,063)	+0.022	0.007	+0.052	< 0.001
AI-tagged, non-bot (5,989)	−0.010	0.628	+0.027	0.138
Zero-AI repos (48,117)	+0.025	0.607	+0.004	0.895

[1]Bonferroni correction adjusts the significance threshold for multiple comparisons. When testing 7 dimensions simultaneously, the probability of at least one false positive at $\alpha = 0.05$ is approximately 30%. Dividing the threshold by the number of tests (0.05 / 7 = 0.007) controls the family-wise error rate.

Subgroup	→ bugs coef	p	→ rework coef	p
Low-AI repos (18,772)	+0.000	0.975	+0.059	< 0.001
High-AI repos (21,163)	+0.023	0.014	+0.042	< 0.001

The pattern is consistent across subgroups. For human-authored PRs, specs are associated with *more* bugs and *more* rework — the confounding-by-indication pattern. For AI-tagged PRs, the coefficients are directionally negative for bugs but not significant ($p = 0.63$), and positive for rework ($p = 0.14$). The null holds whether a repo has zero AI adoption, moderate adoption, or high adoption.

4.6.6 High-Quality Specs Only

A likely reviewer objection is that our `specd` measure is too loose — that linked Jira tickets are not "real" specifications in the SDD sense, and only high-quality structured specs should show the benefit. We test this directly by restricting to top-quartile (quality score ≥ 67) and top-decile (≥ 77) specs. Quality thresholds are computed from the scored subset's distribution (median = 55, p75 = 67, p90 = 77).

Test	→ bugs coef	p	Identifying authors
Top-quartile specs vs. all others	−0.023	0.047	226
Top-decile specs vs. all others	−0.013	0.412	114
High-quality vs. NO spec (low-quality excluded)	−0.019	0.146	201
AI-tagged + high-quality spec	−0.048	0.045	42

The top-quartile result is marginally significant (−2.3pp, $p = 0.047$) but does not survive scrutiny. First, when we perform the cleanest comparison — high-quality specs vs. PRs with *no spec at all*, excluding low-quality specs entirely — the effect is not significant ($p = 0.146$). Second, if there were a real dose-response relationship, top-decile specs should show a *stronger* effect than top-quartile. Instead the effect weakens ($p = 0.412$) — the opposite of what a causal relationship predicts. Third, the top-quartile result would not survive Bonferroni correction across the four tests shown. Fourth, even top-quartile specs still *increase* rework (+2.8pp, $p = 0.078$).

The AI + high-quality spec result ($p = 0.045$, −4.8pp) is the closest test to the agentic SDD workflow vendors are selling. However, it is identified from only 42 authors — far too few for a stable estimate — and the effect vanishes when comparing AI specs broadly ($p = 0.628$). We note it as suggestive but not robust.

The objection that our measure is "too loose" does not rescue the SDD claims. Even restricting to the best-quality specs, the protective effect is fragile, does not follow a dose-response pattern, and does not reduce rework.

4.6.7 JIT Features as Primary Controls

A reviewer concern is that size controls (log additions, deletions, files) are insufficient — the JIT defect prediction framework (Kamei et al., 2013) captures task complexity dimensions (entropy, file age, developer experience) that may confound the specification–defect association. We re-run H1 and H2 with 13 JIT features as additional controls alongside size controls (sexp excluded for zero variance).

Test	Size controls only	1. JIT controls	Change
H1: specs → bugs (coef)	+0.017 ($p = 0.021$)	+0.003 ($p = 0.517$)	−82%
H2: specs → rework (coef)	+0.049 ($p < 0.001$)	+0.033 ($p < 0.001$)	−33%

For H1, the spec coefficient drops 82% and loses significance when JIT features are added, indicating that the apparent spec–defect association is largely explained by task complexity dimensions that size alone does not capture. For H2, the coefficient drops 33% but remains significant: specifications remain associated with more rework even after controlling for JIT risk features, though the effect is attenuated. The within-author spec coefficient in the JIT-controlled model (+0.006, $p = 0.517$) is indistinguishable from zero for defects.

4.6.8 Propensity Score Matching

As an alternative to regression-based controls, we match each spec'd PR to an unspec'd PR with a similar JIT risk profile using nearest-neighbor propensity score matching (caliper = 0.05 SD of the logit propensity score). This directly addresses confounding by indication by comparing spec'd PRs to observationally similar unspec'd PRs.

Outcome	Spec'd (matched)	Unspec'd (matched)	Difference
Bug rate (7,159 pairs)	15.8%	16.4%	−0.6pp ($p = 0.31$)
Rework rate (7,445 pairs)	14.9%	4.4%	+10.5pp ($p < 0.001$)

All 16 covariates are well-balanced after matching (all standardized mean differences < 0.1). For defects, the matched spec'd and unspec'd bug rates are indistinguishable: the raw +2.8pp difference is entirely explained by JIT risk profile differences. For rework, the result is different: spec'd PRs still show 3.2× higher rework even after matching on all observable risk features (only 11% attenuated vs. raw). This suggests that the rework association is not fully explained by task complexity — either specifications mark tasks with unmeasured complexity dimensions, or the specification process itself generates downstream rework (e.g., by creating a documented target for reviewers to request changes against).

4.6.9 Temporal Analysis: The SDD Era

Specification adoption in our dataset is not uniform over time. Monthly specification rates rose from near zero before September 2025 to 14.6% by March 2026, coinciding with the release of Spec Kit (September 2025) and Kiro (July 2025). AI-tagged PR rates rose from 0.9% to 5.6% over

the same period. If SDD tools are driving specification adoption and those specifications improve outcomes, the effect should be visible in the most recent data — the period closest to production SDD usage.

We restrict the within-author analysis to the most recent three months of the observation window (January–March 2026), when specification rates are highest (13.5%) and AI adoption is most prevalent (4.4%):

Test	**Recent 3 months coef**	p	**Full dataset coef**	p
Specs → SZZ bugs	+0.017	0.036	+0.019	0.011
Specs → rework	+0.042	< 0.001	+0.050	< 0.001
AI + specs → bugs	+0.009	0.646	—	—
AI + specs → rework	−0.010	0.588	—	—

The pattern is identical. In the period of highest SDD adoption, specification artifacts are still associated with *more* bugs (+1.7pp) and *more* rework (+4.2pp), not less. For AI-tagged PRs specifically, specs have no effect in either direction. The null result is not a historical artifact — it persists in the data most representative of the SDD era.

A caveat on the temporal data: SZZ-traced bug rates for the most recent months are likely underestimates, because defects introduced recently may not yet have been fixed (and therefore cannot be traced back to their introducing commits). This right-censoring affects absolute bug rates but not the *relative* comparison between spec'd and unspec'd PRs within the same time window, which is the quantity of interest.

The null result holds across every angle: four outcome measures, two predictive frameworks (LPM and JIT), propensity score matching on JIT risk profiles, seven quality dimensions, two units of analysis (PR-level and repo-level), six subgroup cuts, multiple quality thresholds, and temporal restriction to the period of highest SDD adoption. When JIT features are added as controls, the H1 spec coefficient drops 71% and loses significance; propensity score matching eliminates the association entirely. No specification measure robustly predicts fewer defects or less rework under any operationalization.

5 Discussion

5.1 Scale and Power

We analyze 88,052 pull requests across 119 repositories, with 894 authors who have both spec'd and unspec'd work. A 2-percentage-point reduction in bug rate would be detectable with high power given our sample size. The within-author estimates are not merely insignificant but *directionally reversed* across most tests, suggesting this is not a power problem.

However, three caveats apply. First, our specification proxy is imperfect (Section 6): if SDD tools produce qualitatively different specifications from the organic artifacts we measure, the true effect could differ. Second, SZZ measurement noise (46–71% misattribution; (Costa et al., 2017)) reduces statistical power, though non-differential error attenuates toward the null rather than creating reversed associations. Third, residual within-author confounding (task selection) remains possible despite the fixed-effects design.

The strongest candidate for a real effect — top-quartile spec quality reducing bugs by 2.3 percentage points — does not survive the cleanest comparison (high-quality specs vs. no spec, $p = 0.146$) and violates dose-response (top-decile weaker than top-quartile).

5.2 All Roads Lead to Confounding by Indication

The pattern across all five hypotheses is identical: naive analysis suggests an association (sometimes in the claimed direction, sometimes opposite), but within-author analysis reveals this as confounding by indication. Harder, larger, riskier tasks are more likely to receive specifications — and independently more likely to produce defects and require rework.

The same pattern is well-known in clinical medicine as confounding by indication: patients who receive treatment are sicker than those who don't, creating a naive association between treatment and worse outcomes (Salas et al., 1999). The same logic applies here: pull requests that receive specifications are *harder* than those that don't, because developers (rationally) invest specification effort in proportion to task complexity.

JIT risk feature profiles confirm this directly. In the most recent three months (January–March 2026), spec'd PRs have significantly higher change entropy (1.3×, $p < 0.001$), more lines added (1.6×, $p < 0.001$), touch older files (1.8×, $p < 0.001$), and involve more prior developers (1.2×, $p < 0.001$) than unspec'd PRs. Every JIT risk dimension is elevated in spec'd work. Notably, spec'd PR authors have *less* experience than unspec'd PR authors (median 62 vs. 121 prior commits in the repository, $p < 0.001$) — less experienced developers spec more, not less, consistent with specifications being a support mechanism for harder tasks rather than an optimization used by experts.

The implication is that any observational study comparing spec'd to unspec'd work will overestimate the positive association between specifications and poor outcomes (or underestimate any benefit) unless it controls for the endogenous selection of which tasks receive specifications.

5.3 What Actually Predicts Defects

If specifications do not predict defects, what does? The JIT risk features provide a clear answer. In the most recent three months, bug-introducing PRs differ from clean PRs on every code-change dimension: median lines added is 5.5× higher (203 vs. 37, $p < 0.001$), prior changes to the affected files are 3.5× higher (122 vs. 35, $p < 0.001$), file size is 3.1× larger (3,408 vs. 1,091 lines, $p < 0.001$), and change entropy is 1.9× higher ($p < 0.001$). These are structural properties of the change, not properties of the specification.

Developer experience does not protect against defects — it predicts *more* of them. Bug-introducing PR authors have 2.1× more experience than clean PR authors (median 213 vs. 101 prior commits, $p < 0.001$). This is not because experience causes bugs; it is because experienced developers work on harder, riskier code. The same confounding-by-indication pattern that explains the specification association explains the experience association.

AI-tagged PRs show the same defect predictors as human PRs. Within AI-authored work, buggy PRs touch 2× more directories, add 3.7× more lines, and modify files with 3.7× more prior changes than clean AI PRs. AI does not introduce a novel failure mode — it works on riskier code more often (median lines added: 75 for AI vs. 44 for human, $p < 0.001$) and produces bugs at a commensurately higher rate (14.8% vs. 9.3%).

AI bugs are caught faster than human bugs (median 14.6 days to fix vs. 27.0 days, $p < 0.001$), though this is partly a right-censoring artifact: AI adoption is recent, so AI bugs mechanically have shorter maximum possible fix times. AI bug authors are more likely to fix their own bugs (51.9% self-fix rate vs. 41.4%, $p < 0.001$).

Pooled data suggest that AI bugs cascade more: 45% of AI-tagged SZZ bugs also trigger rework, compared to 11.2% of human bugs. However, this does not hold within repositories. In repos with at least 5 AI and 5 human buggy PRs (14 repos), AI rework|buggy is higher in only 7 of 14 (mean $\Delta = -1.6\%$, $p = 0.70$). The pooled difference is driven by repo composition: AI adoption rate correlates $\rho = 0.92$ with repo-level rework rate ($p < 0.001$). AI-heavy repos have high rework rates for all contributors, not just AI-tagged PRs. This is another instance of the confounding-by-indication pattern — repos that adopt AI tools are different kinds of projects, and those projects have higher rework rates independent of AI.

5.4 Review Does Not Mediate the Claimed Effect

A plausible mechanism for specification-driven quality improvement is that specifications improve the review process: reviewers who can compare code against a spec should catch more defects. If this mechanism operated, spec'd PRs should receive more thorough review and, conditional on that review, produce fewer defects.

Spec'd PRs do receive more review. Within-author, the same developer's spec'd PRs take 56% longer to merge (median 23.8 vs. 15.3 hours, $p < 0.001$) and accumulate more review cycles (+0.024 log-cycles, $p < 0.001$). But this additional review does not translate into fewer defects. Within-author, more review cycles predict *more* bugs (+0.026, $p < 0.001$), not fewer — the same confounding pattern: harder PRs receive both more review and more bugs.

AI-tagged PRs receive *less* review scrutiny than human PRs: median time-to-merge is 15.2 hours for AI vs. 17.4 hours for human ($p = 0.002$). This is concerning independently of the specification question — AI-generated code, which has a higher base defect rate (14.8% vs. 9.3%), is being merged faster, not slower.

5.5 What This Study Shows

We test the specific, concrete claims made by SDD tool vendors — that specification artifacts reduce defects, prevent rework, and constrain AI scope — using the closest available proxy: organic specification artifacts scored on the same quality dimensions SDD tools prescribe. Across 88,052 pull requests spanning 119 repositories, the naive association is reversed: specifications accompany *more* defects. After within-author controls, we cannot distinguish the true effect from zero. The most parsimonious interpretation is confounding by indication: harder tasks receive specifications and independently produce more defects, with no residual protective effect visible at our resolution.

This does not prove specifications are useless. It means we cannot detect the claimed benefits at this scale, with this proxy, using the strongest observational design available. Specifications may provide value through mechanisms we cannot measure (team alignment, downstream review quality, audit trails) — but these are not the claims being made. The vendors claim measurably "fewer bugs" and "reduced rework." Our data test these claims and find no support.

The alternative interpretation — that specifications genuinely prevent defects but only for the hardest tasks, and the difficulty effect dominates — does not rescue the marketing claims either. If the benefit exists only for tasks so difficult that their inherent defect propensity overwhelms the specification benefit, then "specifications reduce defects" is misleading at best. The net observable effect is more defects, not fewer.

SDD vendors assert that the specification artifact itself is the causal mechanism — that the spec constrains agent behavior and enables self-validation during generation (Section 3.1). Our results are incompatible with this assertion.

A likely objection is that our data predates true agentic workflows, where the specification is the primary input to an AI agent rather than a human reference document. We address this empirically in our subgroup analysis (AI-tagged PRs show no spec benefit), but there is also a conceptual response. SDD vendors conflate two distinct functions: *directing* the AI (telling it what to build) and *ensuring quality* (fewer bugs, less rework, functional completeness). Specifications may be effective at the first — giving an agent a structured task description is clearly better than a vague prompt. But direction is not quality. An agent faithfully executing a spec will reproduce every gap, ambiguity, and wrong assumption in that spec, with high confidence. A spec can be exhaustively long and still incomplete. It can specify the wrong behavior precisely. It can omit error cases, edge conditions, and integration constraints that the agent will not independently discover, because the agent treats the spec as ground truth. The specification tells the AI *what* to build. It does not tell the AI *what it forgot to specify*. The quality improvement claimed by SDD vendors requires the spec itself to be correct and complete — which is the hard part of software engineering that specifications do not solve, they merely relocate.

5.6 Where Specifications May Still Have Value

Specifications may provide value through mechanisms our study does not measure: audit trails (what the code was *supposed* to do), durable documentation (reducing the knowledge gap as AI generates more code), and team alignment (shared understanding before implementation). These

are organizational benefits, not defect-prevention benefits, and our data does not speak to them. There is also a timing distinction: a specification written *after* iterative validation may function differently from one written *before* any code exists. SDD tools treat the spec as the starting point, but a spec written before validation is a structured guess. Our data cannot distinguish specification-as-input from specification-as-output, but the distinction may matter for future research.

5.7 Vendor Evidence and Construct Gap

No SDD tool vendor has published empirical evidence for the quality claims tested here. The evidence base consists of first-principles reasoning ("clearer input → better output") and developer testimonials. GitHub describes Spec Kit as "an experiment" (Delimarsky, 2025) and publishes no effectiveness data. The third-party evaluations that do exist are negative: Böckeler found Spec Kit's overhead "overkill for the size of the problem" (Böckeler, 2025) Eberhardt found it roughly ten times slower than iterative development (Eberhardt, 2025).

The specification artifacts in our dataset are not generated by SDD tools. A text search of all 100,247 PR descriptions, titles, and file paths for `.specify/`, `.speckit/`, `.kiro/`, and `spec-kit` returns zero matches. The specification adoption we observe — rising from near zero to 14% over 2025–2026 — reflects organic specification behavior: linked issues, design documents, structured PR descriptions. We measure the *kind* of specification content that SDD tools claim to improve upon, scored on the same quality dimensions those tools prescribe, but the artifacts are not tool-generated.

This is a genuine limitation. SDD tools produce specifications in a specific format through a guided workflow, fed directly to an AI agent. Our proxy measures organic specification artifacts scored on the same dimensions. If the tool's format itself is the active ingredient — something beyond the seven quality dimensions we measure — that would be a testable claim. Our data show that the content typically found in specifications, measured on the dimensions SDD tools prescribe, does not predict quality improvement.

An important implication: SDD tooling imposes real costs (specification-writing time, process overhead, organizational change). Teams adopting SDD on the basis of vendor claims are making investment decisions on unsubstantiated assertions. We do not argue that thinking about requirements is wasted effort. We argue that the *products* claiming to reduce defects through specification artifacts have not demonstrated that they do. The best available proxy evidence shows no such effect. Production-scale data from purpose-built SDD tools does not yet exist in the public domain, and vendors with the resources to produce it have not done so.

6 Threats to Validity

6.1 Are We Measuring the Right Thing?

Three gaps separate what we measure from what SDD vendors claim.

We detect specification artifacts, not specification processes. Our classifier flags linked issues, structured descriptions, and referenced design documents. A linked Jira ticket created to satisfy a process requirement counts the same as a detailed design document written after iterative refinement. If the benefit comes from the *thinking* rather than the *artifact*, our measure is too coarse. Two observations partially address this concern. First, the within-author design compares the same developer's spec'd and unspec'd work, controlling for that person's consistent habits. Second, H3 and H4 test quality *within* spec'd PRs on seven SDD-aligned dimensions. If deeper thinking produced better artifacts, those artifacts should predict better outcomes. They do not. The remaining objection — that process depth helps through a mechanism invisible in the artifact itself — is difficult to test with any observational design.

We test organic specifications, not SDD tool output. Spec Kit generates structured documents through a guided workflow and feeds them to a coding agent. A linked GitHub issue is not the same thing. Our operationalization is the broadest reasonable proxy available, but it may miss whatever specific mechanism the tools provide. The high-quality spec subsample (top-quartile, scored on the dimensions SDD tools prescribe) is the closest approximation. It too shows no robust effect.

We cannot observe agentic workflows directly. Even for AI-tagged PRs with specifications, we do not know whether the spec was actually used as input to an agent or just written for human consumption. True agentic SDD — where the agent reads the spec, generates code from it, and validates against it — is not observable in our data. The 42-author AI + high-quality spec subsample is the closest proxy we have.

6.2 Are We Measuring Outcomes Accurately?

Defect detection is noisy. The SZZ algorithm traces fix commits back to their introducing commits via `git blame`. da Costa et al. (Costa et al., 2017) found that basic SZZ misattributes 46–71% of bug-introducing changes. We use the basic variant for implementation simplicity across 103 repositories. This noise is substantial, but we expect it to be non-differential — there is no reason SZZ accuracy would vary with specification status. Non-differential measurement error attenuates true associations toward the null, reducing power but not creating spurious reversed effects. Small true effects may be undetectable in our data.

Quality scoring is automated and unvalidated. Our LLM-based rubric (Claude Haiku) measures *formal* quality — whether the text contains outcome descriptions, error states, and acceptance criteria — not *functional* quality. A spec can score highly and still describe the wrong behavior. The rubric's seven dimensions align with what SDD tools prescribe, which means our scoring shares whatever blind spots those templates have. The scoring was applied only to PRs with substantial description text, creating a non-random subsample.

AI detection relies on voluntary tagging. We identify AI-assisted PRs via self-reported markers in the PR body. Developers who use AI without attribution are invisible. The false-negative rate is unknown. If many untagged PRs involved AI, the H5 comparison would be diluted toward the null.

The control group may be contaminated. Some "unspec'd" PRs may have had specifications shared verbally, in Slack, or in external tools. This would attenuate any treatment effect. However, if informal specification produces equivalent outcomes to formal artifacts, the SDD value proposition is undermined: vendors claim the *tooling and artifact* add value, not that thinking about requirements helps in general.

6.3 Could Something Else Explain the Results?

Confounding by indication is the primary threat. Developers spec hard tasks. Hard tasks produce bugs. The within-author design addresses this by comparing each developer to themselves, but within-author task selection remains possible: if an author consistently specs their hardest work and skips specs on easy tasks, the comparison is biased. The JIT-controlled models (Section 4.7) and propensity score matching (Section 4.8) address this further, and both confirm the null for defects.

Time-varying confounding. An author's skill, tooling, and project context may change over the observation period. The within-author estimator assumes these are stable.

6.4 Can These Results Generalize?

Open-source only. All 119 repositories are open-source. SDD tools target commercial teams with product managers, QA processes, and sprint planning. The specification practices and defect economics may differ.

Convenience sample. Repositories were not randomly selected. Results cannot be generalized beyond the sample.

Pre-agentic era. Most PRs predate widespread agentic AI coding tools. The dataset contains 5,989 AI-tagged PRs (after bot exclusion), the closest available proxy for agentic workflows. Within this subsample, specs show no effect on bugs ($p = 0.98$) or rework ($p = 0.33$). We cannot rule out that true agentic workflows would produce different results — but that is itself an untested claim.

6.5 Statistical Caveats

Multiple comparisons. We test five hypotheses without correction. At $\alpha = 0.05$, approximately 0.25 false positives are expected by chance.

Power for AI interaction. The scope-constraint interaction (H5) is identified from 202 authors with variation in both specification and AI use, sufficient for the significant result obtained ($p = 0.001$).

7 Conclusion

Spec-driven development tools make five specific, testable claims: specifications reduce defects (H1), prevent rework (H2), improve outcomes through higher-quality requirements (H3, H4), and constrain AI-generated code scope (H5). Production-scale data from purpose-built SDD tools is not yet publicly available. We test the claims using the best available proxy: 88,052 pull requests across 119 open-source repositories, where specification artifacts are scored on the same quality dimensions that SDD tools prescribe, using within-author fixed-effects estimation.

Four of five hypotheses are not supported. The naive association between specifications and defects is reversed (specifications accompany *more* defects); after within-author controls, the most parsimonious interpretation is confounding by indication. Specification quality has no meaningful effect on rework ($p = 0.687$). One claim finds partial support: specifications constrain AI-generated code scope ($p = 0.001$), but this scope constraint does not translate into fewer defects or less rework. Nine robustness checks confirm the quality null. Adding specification presence to a JIT defect model contributes Δ Pseudo $R^2 = 0.00003$ ($p = 0.250$). Specification presence proxies for task complexity, not for quality improvement.

Three important caveats bound these findings. First, we test organic specification artifacts, not SDD tool-generated specifications — the construct gap is real (Section 6). Second, SZZ defect tracing has substantial measurement noise (46–71% misattribution), which attenuates true effects. Third, our open-source convenience sample may not generalize to commercial teams. Whether purpose-built SDD tooling would produce different results on commercial codebases remains an open question. The best available proxy evidence offers no support for the quality claims being made.

The dataset and analysis code are available at: `github.com/brennhill/delivery-gap-research`.

8 References

Angrist, J. D., & Pischke, J.-S. (2009). *Mostly Harmless Econometrics: An Empiricist's Companion*. Princeton University Press.

Beck, K., & al. (2001). *Manifesto for Agile Software Development*.

Boehm, B. W. (1981). *Software Engineering Economics*. Prentice Hall.

Brooker, M. (2025, July 15). *Kiro and the Future of AI Spec-Driven Software Development*. https://kiro.dev/blog/kiro-and-the-future-of-software-development/

Böckeler, B. (2025, October 15). *Understanding Spec-Driven-Development: Kiro, spec-kit, and Tessl*. https://martinfowler.com/articles/exploring-gen-ai/sdd-3-tools.html

Costa, D. A. da, McIntosh, S., Shang, W., Kuber, U., Coelho, R., & Hassan, A. E. (2017). A Framework for Evaluating the Results of the SZZ Approach for Identifying Bug-Introducing Changes. *IEEE Transactions on Software Engineering*, *43*(7), 641–657.

Delimarsky, D. (2025, September 2). *Spec-Driven Development with AI: Get Started with a New Open Source Toolkit*. https://github.blog/ai-and-ml/generative-ai/spec-driven-development-with-ai-get-started-with-a-new-open-source-toolkit/

Eberhardt, C. (2025, November 26). *Putting Spec Kit Through Its Paces: Radical Idea or Reinvented Waterfall?*. https://blog.scottlogic.com/2025/11/26/putting-spec-kit-through-its-paces-radical-idea-or-reinvented-waterfall.html

Femmer, H., Fernández, D. M., Wagner, S., & Eder, S. (2017). Rapid Quality Assurance with Requirements Smells. *Journal of Systems and Software*, *123*, 190–213.

GitHub Copilot Academy. (2025, September). *Spec-Driven Development*. https://copilot-academy.github.io/workshops/immersive-experience/spec_driven_development

Griffin, L., & Carroll, R. (2026, January 12). *Spec Driven Development: When Architecture Becomes Executable*. https://www.infoq.com/articles/spec-driven-development/

Guo, H., Ji, Y., & Zhou, X. (2025). Sedeve-Kit, a Specification-Driven Development Framework for Building Distributed Systems. *Arxiv Preprint*. https://arxiv.org/abs/2509.11566

Kamei, Y., Shihab, E., Adams, B., Hassan, A. E., Mockus, A., Sinha, A., & Ubayashi, N. (2013). A Large-Scale Empirical Study of Just-in-Time Quality Assurance. *IEEE Transactions on Software Engineering*, *39*(6), 757–773.

Marri, S. R. (2026). Constitutional Spec-Driven Development: Enforcing Security by Construction in AI-Assisted Code Generation. *Arxiv Preprint*. https://arxiv.org/abs/2602.02584

Mund, J., Femmer, H., Fernández, D. M., & Eckhardt, J. (2015). Does Quality of Requirements Specifications Matter? Combined Results of Two Empirical Studies. *Proceedings of the 9th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, 1–10.

Naszcyniec, R. (2025, October 22). *How Spec-Driven Development Improves AI Coding Quality*. https://developers.redhat.com/articles/2025/10/22/how-spec-driven-development-improves-ai-coding-quality

Neyman, J., & Scott, E. L. (1948). Consistent Estimates Based on Partially Consistent Observations. *Econometrica*, *16*(1), 1–32.

Piskala, D. B. (2026). Spec-Driven Development: From Code to Contract in the Age of AI Coding Assistants. *Arxiv Preprint*. https://arxiv.org/abs/2602.00180

Roy, Y. (2026). The Kitchen Loop: User-Spec-Driven Development for a Self-Evolving Codebase. *Arxiv Preprint*. https://arxiv.org/abs/2603.25697

Salas, M., Hofman, A., & Stricker, B. H. C. (1999). Confounding by Indication: An Example of Variation in the Use of Epidemiologic Terminology. *American Journal of Epidemiology*, *149*(11), 981–983.

Shull, F., Basili, V., Boehm, B., Brown, A. W., Costa, P., Lindvall, M., Port, D., Rus, I., Tesoriero, R., & Zelkowitz, M. (2002). What We Have Learned About Fighting Defects. *Proceedings of the 8th International Symposium on Software Metrics*, 249–258.

Yanchuleff, R. (2025, July 15). *From Chat to Specs: A Deep Dive into AI-Assisted Development with Kiro*. https://kiro.dev/blog/from-chat-to-specs-deep-dive/

Zaninotto, F. (2025, November 12). *Spec-Driven Development: The Waterfall Strikes Back*. https://marmelab.com/blog/2025/11/12/spec-driven-development-waterfall-strikes-back.html

Zietsman, C. (2026). The Specification as Quality Gate: Three Hypotheses on AI-Assisted Code Review. *Arxiv Preprint*. https://arxiv.org/abs/2603.25773

Śliwerski, J., Zimmermann, T., & Zeller, A. (2005). When Do Changes Induce Fixes?. *Proceedings of the 2nd International Workshop on Mining Software Repositories (MSR)*, 1–5.